Compiler Design Lab Report 6
Pseudo Assembly Code Generation Using YACC

Abdullah Al Jubaer Gem
ID: 210041226
Section: 2B

August 16, 2026

1 Objective

Develop a YACC program that generates Pseudo Assembly Code (PAC) for assignment statements and arithmetic expressions, using Syntax-Directed Definitions (SDDs) to drive code generation as the parser reduces each production.

2 Grammar

Operator precedence and associativity are encoded structurally: addition/subtraction sit above multiplication/division/modulo, which sit above parenthesised and atomic factors.

$$
\begin{aligned}
\text{Program} &\rightarrow \text{StmtList} \\
\text{StmtList} &\rightarrow \text{StmtList Stmt} \mid \text{Stmt} \\
\text{Stmt} &\rightarrow \text{id} = \text{Expr;} \\
\text{Expr} &\rightarrow \text{Expr} + \text{Term} \mid \text{Expr} - \text{Term} \mid \text{Term} \\
\text{Term} &\rightarrow \text{Term} * \text{Factor} \mid \text{Term/Factor} \mid \text{Term \% Factor} \mid \text{Factor} \\
\text{Factor} &\rightarrow \text{(Expr)} \mid \text{id} \mid \text{number}
\end{aligned}
$$

3 Syntax-Directed Definition

Every non-terminal that produces a value (`Expr`, `Term`, `Factor`) carries a synthesized attribute `place` – implemented as the YACC semantic value `$$` – holding the name of the variable, temporary, or literal that stores its computed result.

- **Expr → Expr1 + Term**: `Expr.place = newTemp();` emit `ADD Expr.place, Expr1.place, Term.place`.
- **Expr → Expr1 - Term**: analogous, using `SUB`.
- **Term → Term1 * Factor**: `Term.place = newTemp();` emit `MUL Term.place, Term1.place, Factor.place` (similarly `DIV` and `MOD` for `/` and `%`).
- **Factor → (Expr)**: `Factor.place = Expr.place` – no new temporary is needed.
- **Factor → id | number**: `Factor.place` is the lexeme text itself.
- **Stmt → id = Expr ;**: emit `MOV id, Expr.place`.

Each rule that introduces a new temporary calls `newTemp()`, which returns sequentially numbered names `T1, T2, T3, ...`, and `emit()`, which writes one instruction line to the output file. Because these actions fire bottom-up as YACC reduces each production, the left operand of an operator is always fully evaluated (and assigned its temporary) before the right operand is visited, and the top-level operator last – which is exactly why the temporary numbering in the sample output below proceeds strictly left-to-right, subexpression by subexpression.

4 Implementation

4.1 FLEX Lexer

```
%{
#include "y.tab.h"
#include <stdlib.h>
#include <string.h>
%}

%option noyywrap

%%

[ \t\r]+                ;
\n                       ;

[0-9]+ {
    yylval.sval = strdup(yytext);
    return NUMBER;
}

[a-zA-Z_][a-zA-Z0-9_]* {
    yylval.sval = strdup(yytext);
    return ID;
}

"+"     return '+';
"-"     return '-';
"*"     return '*';
"/"     return '/';
"%"     return '%';
"="     return '=';
";"     return ';';
"("     return '(';
")"     return ')';

.       return yytext[0];

%%
```

Listing 1: FLEX Lexer (lexer.l)

4.2 YACC Parser

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex(void);
void yyerror(const char *s);
extern FILE *yyin;
extern int yylineno;

FILE *out;
int tempCount = 0;

char* newTemp()
{
    char *buf = malloc(16);
    sprintf(buf, "T%d", ++tempCount);
    return buf;
}

void emit(const char *op, const char *dest, const char *op1, const char *op2)
{
    fprintf(out, "%s %s, %s, %s\n", op, dest, op1, op2);
}
%}

%union {
```

```
    char *sval;
}

%token <sval> ID NUMBER
%type  <sval> Expr Term Factor

%left '+' '-'
%left '*' '/' '%'

%%

Program:
      StmtList
    ;

StmtList:
      StmtList Stmt
    | Stmt
    ;

Stmt:
      ID '=' Expr ';'
        {
            fprintf(out, "MOV %s, %s\n", $1, $3);
        }
    ;

Expr:
      Expr '+' Term  { $$ = newTemp(); emit("ADD", $$, $1, $3); }
    | Expr '-' Term  { $$ = newTemp(); emit("SUB", $$, $1, $3); }
    | Term           { $$ = $1; }
    ;

Term:
      Term '*' Factor { $$ = newTemp(); emit("MUL", $$, $1, $3); }
    | Term '/' Factor { $$ = newTemp(); emit("DIV", $$, $1, $3); }
    | Term '%' Factor { $$ = newTemp(); emit("MOD", $$, $1, $3); }
    | Factor           { $$ = $1; }
    ;

Factor:
      '(' Expr ')' { $$ = $2; }
    | ID           { $$ = $1; }
    | NUMBER       { $$ = $1; }
    ;

%%

void yyerror(const char *s)
{
    fprintf(stderr, "Syntax Error: %s\n", s);
}

int main(int argc, char **argv)
{
    FILE *fp = fopen(argc > 1 ? argv[1] : "input.txt", "r");
    out = fopen("output.txt", "w");

    if (!fp || !out)
    {
        fprintf(stderr, "Error: could not open input/output file.\n");
        return 1;
    }

    yyin = fp;
    yyparse();

    fclose(fp);
    fclose(out);
    return 0;
}
```

Listing 2: YACC Parser (parser.y)

5 Compilation Steps

```
bison -y -d parser.y
flex lexer.l
gcc lex.yy.c y.tab.c -o pacgen
./pacgen
```

Listing 3: Build and Run

6 Sample Input and Output

6.1 Input (`input.txt`)

```
a = b + c;
x = (a+b)*c;
y = a*b + c*d;
```

Listing 4: Sample Input

6.2 Generated Pseudo Assembly Code (`output.txt`)

```
ADD T1, b, c
MOV a, T1
ADD T2, a, b
MUL T3, T2, c
MOV x, T3
MUL T4, a, b
MUL T5, c, d
ADD T6, T4, T5
MOV y, T6
```

Listing 5: Generated Output

7 Conclusion

The YACC-based translator successfully converts assignment statements containing arithmetic expressions into Pseudo Assembly Code, preserving operator precedence and associativity through the grammar structure and generating sequentially numbered temporary variables for every intermediate result. The synthesized `place` attribute, propagated bottom-up through semantic actions, cleanly separates parsing from code emission and produces output identical in structure to the instruction set specified in the lab manual.